

Tipos de datos en SQL

El modelo ya está diseñado y normalizado. Ahora toca llevarlo a código real. Y antes de escribir la primera instrucción, hay algo que necesitas entender: en SQL, cada columna de una tabla tiene un tipo de dato, y ese tipo le dice al motor exactamente qué clase de información puede recibir.

Esto no es un detalle técnico menor. Una columna con el tipo de dato equivocado puede hacer cálculos incorrectos sin avisarte, ocupar el doble del espacio necesario, o hacer que una búsqueda falle sin dar ningún mensaje de error. Elegirlo bien toma segundos. Corregirlo cuando la base ya tiene datos puede tomar horas.

En esta clase trabajamos con seis tipos de datos que cubren el 90 % de los casos que vas a encontrar.

Antes de empezar: ¿qué es un tipo de dato?

Piénsalo así: cuando alguien llena un formulario en papel, hay campos que solo aceptan números, otros que solo aceptan texto, y otros que piden una fecha en un formato específico. Si alguien escribe letras en el campo de teléfono, algo está mal.

En una base de datos funciona igual. Cuando defines una columna, le dices de qué tipo es. A partir de ese momento, el motor solo acepta datos de ese tipo en esa columna. Si intentas meter texto donde hay un número, el motor lo rechaza. Eso es exactamente lo que quieres: que los datos entren bien desde el principio.

Los seis tipos de datos que necesitas

1. INTEGER — para números sin decimales

Cuando el valor es un número entero, sin punto decimal, usas INTEGER. Identificadores, cantidades, conteos: todo eso va aquí.

SQL

```
SELECT 1500 AS numero_entero
```

¿Por qué no usar siempre el mismo tipo numérico para todo? Porque INTEGER ocupa menos espacio y es más rápido de procesar. Si sabes que un campo nunca va a tener decimales, no tiene sentido darle esa capacidad.

2. NUMERIC — para números con decimales

Precios, porcentajes, medidas que necesitan precisión: para eso existe NUMERIC. Pero tiene una particularidad: hay que decirle cuántos dígitos puede tener en total y cuántos de esos van después del punto decimal.

SQL

```
SELECT CAST(1500 AS NUMERIC(10,2)) AS numero_decimal
```

NUMERIC(10, 2) se lee así: hasta 10 dígitos en total, de los cuales 2 van después del punto. Resultado: **1500.00**.

Dos cosas importantes que pasan con NUMERIC:

- Si el número tiene más decimales de los definidos, el motor lo **redondea** automáticamente. Por ejemplo, **0.00750** con **NUMERIC(10, 4)** se guarda como **0.0075**.
- Si el número supera el total de dígitos permitidos, el motor devuelve un **error**. Si defines **NUMERIC(10, 2)** y tratas de guardar un número de 11 dígitos, no va a pasar.

¿Qué es CAST? Es un comando que convierte un valor de un tipo de dato a otro. En el ejemplo de arriba, convierte el número entero **1500** en un NUMERIC con decimales. Lo vas a usar con frecuencia.

3. TEXT — para texto sin límite

Nombres, descripciones, comentarios, cualquier contenido de texto libre: todo eso va en TEXT. No necesitas definir un largo máximo.

SQL

```
SELECT 'Laptop HP ProBook 450' AS nombre_producto
```

Una regla que no cambia en SQL: **el texto siempre va entre comillas simples**. No dobles, simples. Cualquier valor escrito así es tratado automáticamente como texto.

4. VARCHAR(n) — para texto con límite definido

A veces sabes de antemano que un campo no puede superar cierta cantidad de caracteres. Un código de país siempre tiene 3 letras. Un número de documento tiene un máximo. Datos que vienen de formularios con restricciones. Para eso existe VARCHAR, y entre paréntesis le dices cuál es el límite.

SQL

```
SELECT CAST('Laptop HP ProBook 450' AS VARCHAR(10)) AS
texto_limitado
```

Resultado: **Laptop HP P**. El texto queda cortado exactamente en el carácter número 10. VARCHAR no avisa, no redondea, simplemente corta. Por eso es clave saber de dónde vienen los datos antes de definir ese límite.

¿Cuándo usar TEXT y cuándo VARCHAR? Si el largo es impredecible o variable, TEXT. Si sabes que el campo tiene un máximo claro, VARCHAR.

5. BOOLEAN — para verdadero o falso

Hay información que solo puede tener dos estados: sí o no, activo o inactivo, entregado o pendiente. Para eso existe BOOLEAN.

SQL

```
SELECT TRUE AS asistio, FALSE AS no_asistio
```

El motor devuelve **T** para verdadero y **F** para falso. En la práctica, piénsalo como la casilla de verificación de un formulario: cuando está marcada es TRUE, cuando está vacía es FALSE.

Se usa mucho para estados: ¿el pedido fue entregado? ¿el usuario está activo? ¿el producto está disponible? Cualquier pregunta que se responde con sí o no es un candidato para BOOLEAN.

6. DATE y TIMESTAMP — para fechas y fechas con hora

Aquí hay una distinción que importa: no es lo mismo saber que algo pasó el 31 de julio de 2024, que saber que pasó el 31 de julio de 2024 a las 10:30:25.

DATE guarda solo la fecha, sin hora. Fecha de nacimiento, fecha de un pedido, fecha de vencimiento.

SQL

```
SELECT CURRENT_DATE AS fecha_hoy
```

```
SELECT '2024-07-31'::DATE AS fecha_pasada
```

CURRENT_DATE devuelve la fecha del servidor en ese momento. La notación **::DATE** es otra forma de decirle al motor cómo interpretar ese texto — equivalente al **CAST**, pero más corta.

TIMESTAMP guarda fecha y hora juntas, hasta el nivel de segundos. Se usa cuando el momento exacto importa: una transacción, el ingreso de un usuario al sistema, la hora exacta de un envío.

SQL

```
SELECT NOW() AS momento_actual
```

```
SELECT '2024-03-15 10:30:25'::TIMESTAMP AS timestamp_ejemplo
```

NOW() devuelve la fecha y hora exacta del servidor en ese instante. El resultado que ves incluye año, mes, día, hora, minuto y segundo.

¿Cuándo usar **DATE** y cuándo **TIMESTAMP**? Cuando solo importa el día, **DATE**. Cuando importa el momento exacto, **TIMESTAMP**.

Una tabla para tenerlos a mano

Tipo de dato	Lo usas cuando...	Ejemplo de valor
INTEGER	El número nunca tiene decimales	1, 42, 1500
NUMERIC(p,d)	Necesitas decimales controlados	1500.00, 0.0075
TEXT	El texto puede tener cualquier largo	'Laptop HP ProBook 450'
VARCHAR(n)	El texto tiene un máximo definido	'COL', 'ARG'
BOOLEAN	El valor solo puede ser sí o no	TRUE, FALSE
DATE	Solo importa el día	2024-07-31
TIMESTAMP	Importa el momento exacto	2024-03-15 10:30:25

El error más común con tipos de datos

Guardar números como texto. Parece inofensivo, pero trae problemas reales: no puedes hacer cálculos con ellos, los filtros no funcionan como esperas y las comparaciones dan resultados extraños. Si un campo es un número, defínelo como número desde el principio, aunque los datos vengan originalmente en formato texto.

Con estos seis tipos de datos tienes lo que necesitas para el siguiente paso: escribir las instrucciones que crean las tablas dentro de la base de datos, columna por columna, con el tipo correcto para cada una.